COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 Data Structure and Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 07/24/26 | **DATE SUBMITTED:** | 07/30/26 |
| **NAME:** | Bungag, Marc Jhowell M. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

## HOA 3.1 Linked Lists

### 1. PROCEDURE OUTPUT

What procedures did you perform?

Screenshot

HOA 3.1 > main-1.cpp > main()

```cpp
#include<iostream>
#include "singly_ll.h"

int main (){
    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;

    head = new SingleList<char>;
    second = new SingleList<char>;
    third = new SingleList<char>;
    fourth = new SingleList<char>;

    head->data = 'M';
    second->data = 'A';
    third->data = 'R';
    fourth->data = 'C';

    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = nullptr;

    std::cout<<"Testing of Traversal:\n";
    ListTraversal(head);

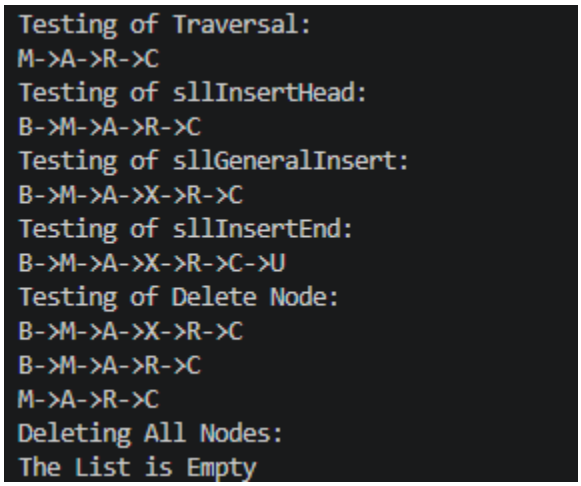
    std::cout<<"Testing of sllInsertHead:\n";
    sllInsertHead('B', &head);
    ListTraversal(head);

    std::cout<<"Testing of sllGeneralInsert:\n";
    sllGeneralInsert('X', head->next->next);
    ListTraversal(head);

    std::cout<<"Testing of sllInsertEnd:\n";
    sllInsertEnd('U', &head);
    ListTraversal(head);

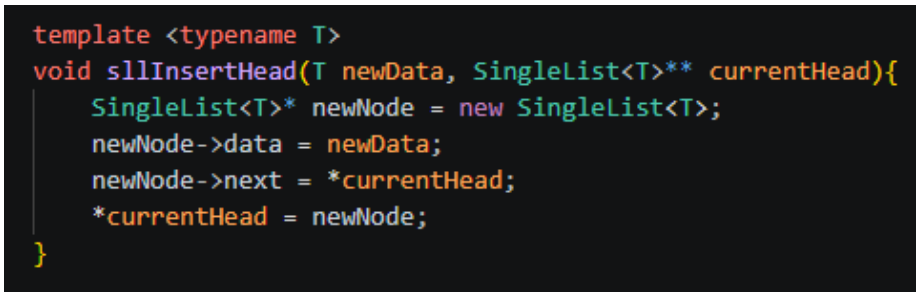
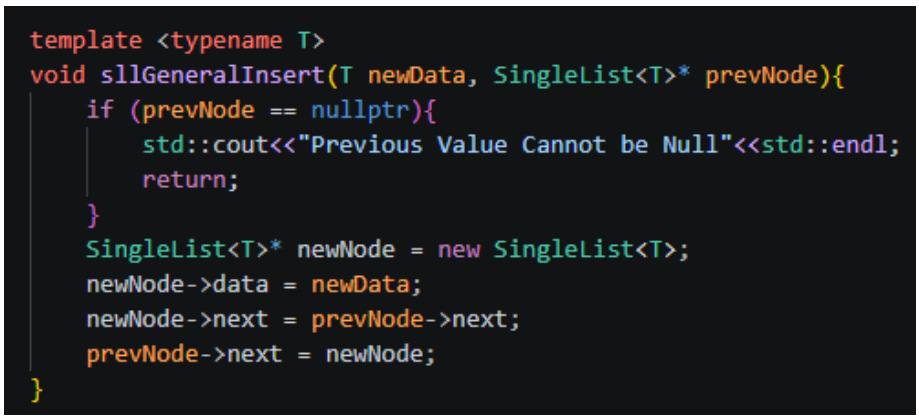
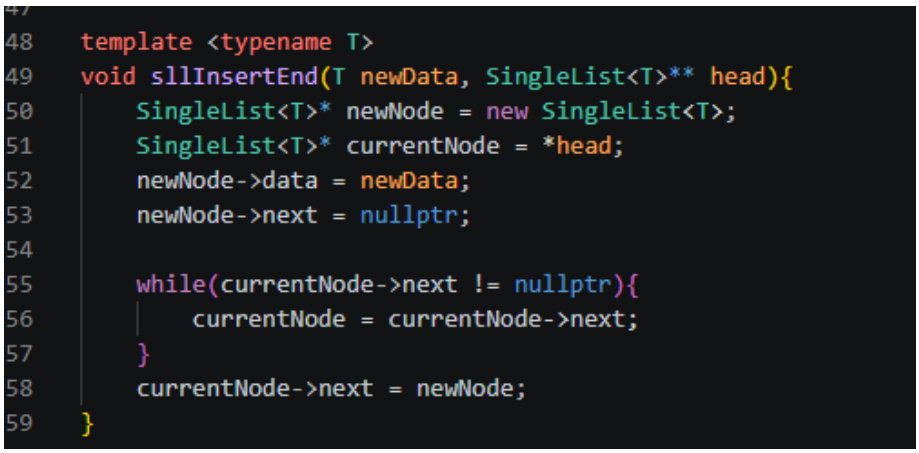

    std::cout<<"Testing of Delete Node:\n";
    sllDeleteNode('U', &head);
    ListTraversal(head);
    sllDeleteNode('X', &head);
    ListTraversal(head);
    sllDeleteNode('B', &head);
    ListTraversal(head);

    std::cout<<"Deleting All Nodes:\n";
    sllDeleteList(&head);
    ListTraversal(head);
    return 0;
}
```

| | |
|---|---|
| |  |
| Discussion | This code creates and connects each node, making it easier to visualize how a linked list is stored in memory. However, this approach is not practical for real-world use because linked lists are typically built and managed dynamically using functions such as insert and traverse. |

| Operation | Screenshot |
|---|---|
| Traversal |  |

```
11  template <typename T>
12  void ListTraversal(SingleList<T>* head){
13      if(head == nullptr){
14          std::cout<<"The List is Empty";
15          return;
16      }
17      while(head != nullptr){
18          std::cout<<head->data;
19          if(head->next != nullptr){
20              std::cout<<"->";
21          }
22          head = head->next;
23      }
24      std::cout<<std::endl;
25
26  }
```

| | |
|---|---|
| Insertion at head |  |
| Insertion at any part of the list |  |
| Insertion at the end |  |

```
template <typename T>
void sllInsertHead(T newData, SingleList<T>** currentHead){
    SingleList<T>* newNode = new SingleList<T>;
    newNode->data = newData;
    newNode->next = *currentHead;
    *currentHead = newNode;
}
```

```
template <typename T>
void sllGeneralInsert(T newData, SingleList<T>* prevNode){
    if (prevNode == nullptr){
        std::cout<<"Previous Value Cannot be Null"<<std::endl;
        return;
    }
    SingleList<T>* newNode = new SingleList<T>;
    newNode->data = newData;
    newNode->next = prevNode->next;
    prevNode->next = newNode;
}
```

```
48  template <typename T>
49  void sllInsertEnd(T newData, SingleList<T>** head){
50      SingleList<T>* newNode = new SingleList<T>;
51      SingleList<T>* currentNode = *head;
52      newNode->data = newData;
53      newNode->next = nullptr;
54
55      while(currentNode->next != nullptr){
56          currentNode = currentNode->next;
57      }
58      currentNode->next = newNode;
59  }
```

| | |
|---|---|
| Deletion of a node |  |

```
template <typename T>
void sllDeleteNode(T findData, SingleList<T>** head){
    if(*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }
    if(currNode == nullptr){
    std::cout<<"The Data was Not Found \n"<<std::endl;
    return;
    }
    if(prevNode == nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

a. Source Code

HOA 3.1 > main-1.cpp > main()

```cpp
#include<iostream>
#include "singly_ll.h"

int main (){
    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;

    head = new SingleList<char>;
    second = new SingleList<char>;
    third = new SingleList<char>;
    fourth = new SingleList<char>;

    head->data = 'M';
    second->data = 'A';
    third->data = 'R';
    fourth->data = 'C';

    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = nullptr;

    std::cout<<"Testing of Traversal:\n";
    ListTraversal(head);
```

Console

```
Testing of Traversal:
M->A->R->C
```

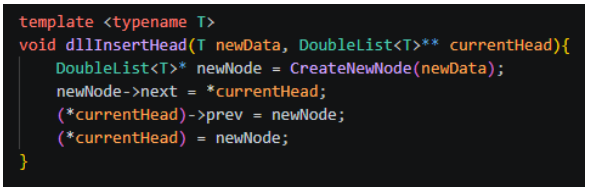
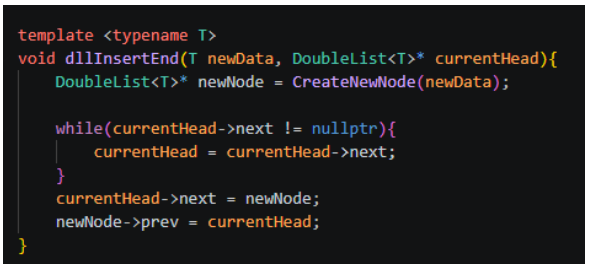
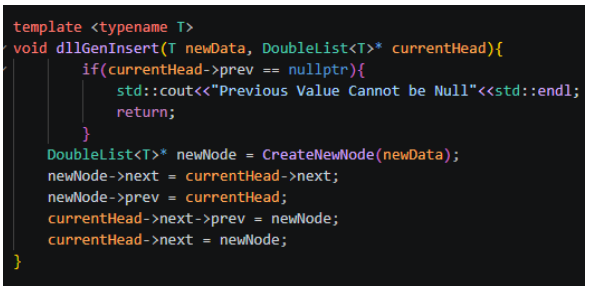
b. Source Code

```cpp
    std::cout<<"Testing of sllInsertHead:\n";
    sllInsertHead('B', &head);
    ListTraversal(head);
```

Console

```
Testing of sllInsertHead:
B->M->A->R->C
```

| | | |
|---|---|---|
| c. | Source Code | `std::cout<<"Testing of sllGeneralInsert:\n";`<br>`sllGeneralInsert('X', head->next->next);`<br>`ListTraversal(head);` |
| | Console | `Testing of sllGeneralInsert:`<br>`B->M->A->X->R->C` |
| d. | Source Code | `std::cout<<"Testing of sllInsertEnd:\n";`<br>`sllInsertEnd('U', &head);`<br>`ListTraversal(head);` |
| | Console | `Testing of sllInsertEnd:`<br>`B->M->A->X->R->C->U` |
| e. | Source Code | `std::cout<<"Testing of Delete Node:\n";`<br>`sllDeleteNode('U', &head);`<br>`ListTraversal(head);`<br>`sllDeleteNode('X', &head);`<br>`ListTraversal(head);`<br>`sllDeleteNode('B', &head);`<br>`ListTraversal(head);` |
| | Console | `Testing of Delete Node:`<br>`B->M->A->X->R->C`<br>`B->M->A->R->C`<br>`M->A->R->C` |
| f. | Source Code | `std::cout<<"Deleting All Nodes:\n";`<br>`sllDeleteList(&head);`<br>`ListTraversal(head);`<br>`return 0;` |
| | Console | `Deleting All Nodes:`<br>`The List is Empty` |

| Screenshot | Analysis |
| --- | --- |
|  | The dllTraverse() function displays the elements of the doubly linked list in both forward and backward directions. It first checks if the list is empty, then traverses from the head to the tail before printing the nodes in reverse order using the prev pointers. |
|  | The dllInsertHead() function inserts a new node at the beginning of the list by updating the necessary pointers and making the new node the head. |
|  | The dllInsertEnd() function inserts a new node at the end of the list by traversing to the last node and linking the new node to it. |
|  | The dllGenInsert() function inserts a new node after the current node by updating the surrounding pointers. |

```
template <typename T>
void dllDelNode(T findData, DoubleList<T>** head){
    if (*head == nullptr) return;

    DoubleList<T>* currNode = *head;

    while(currNode != nullptr && currNode->data != findData){
        currNode = currNode->next;
    }
    if(currNode == nullptr){
    std::cout<<"The Data was Not Found \n"<<std::endl;
    return;
    }
    if(currNode->prev == nullptr){
        *head = currNode->next;
    }else{
        currNode->prev->next = currNode->next;
    }
    if(currNode->next != nullptr){
        currNode->next->prev = currNode->prev;
    }

    delete currNode;
}
#endif
```

The dllDelNode() function removes the node containing the specified data by updating the neighboring pointers and deleting the node.

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

playlist_ll.h

HOA 3.1 > C playlist_ll.h > InsertNewSong<T>(T, PlayList<T>*)

```h
#ifndef PLAYLIST_ll_h
#define PLAYLIST_ll_h

template <typename T>
class PlayList{
    public:
    T data;
    PlayList<T>* prev = nullptr;
    PlayList<T>* next = nullptr;
};

//creates the playlist
template <typename T>
PlayList<T> *CreatePlayList(T newData){
    PlayList<T> *newSong = new PlayList<T>;

    newSong->data = newData;
    newSong->next = nullptr;
    newSong->prev = nullptr;
    return newSong;
}

//prints the playlist or the traverse
template <typename T>
void DisplayPlaylist(PlayList<T>* currentSong){
    if(currentSong == nullptr){
        std::cout<<"The Playlist is Empty"<<std::endl;
        return;
    }
     PlayList<T>* startingSong = currentSong;

    do{
        std::cout << currentSong->data << std::endl;
        currentSong = currentSong->next;

    }while(currentSong != startingSong);

    std::cout << "\n";
}

//every new song is always added in the last part of the playlist, so we use insert end
template <typename T>
void InsertNewSong(T newData, PlayList<T>* currentSong){
    PlayList<T>* newSong = CreatePlayList(newData);
    PlayList<T>* endSong = currentSong->prev;
    endSong->next = newSong;
    newSong->prev = endSong;

    newSong->next = currentSong;
    currentSong->prev = newSong;
}
```

```
52
53  //delete a song
54  template <typename T>
55  void RemoveSong(T findData, PlayList<T>** head){
56      if(*head == nullptr) return;
57
58      PlayList<T>* currentSong = *head;
59
60      do{
61          if(currentSong->data == findData)
62              break;
63          currentSong = currentSong->next;
64      }while(currentSong != *head);
65      if(currentSong->data != findData){
66          std::cout << "Song is Not Included in the Playlist\n";
67          return;
68      }
69      if(currentSong->next == currentSong){
70          *head = nullptr;
71      }
72      else{
73          currentSong->prev->next = currentSong->next;
74          currentSong->next->prev = currentSong->prev;
75          if(currentSong == *head){
76              *head = currentSong->next;
77          }
78      }
79      delete currentSong;
80  }
81
82  //next song
83  template <typename T>
84  void PlayNextSong(PlayList<T>** currentSong){
85
86      if(*currentSong == nullptr){
87          std::cout<<"The Playlist is Empty"<<std::endl;
88          return;
89      }
90      *currentSong = (*currentSong)->next;
91          std::cout<<"You are Already at the Last Song"<<std::endl;
92          std::cout<<"Now Playing: "<<(*currentSong)->data<<std::endl;
93  }
94
95  //play previous song
96  template <typename T>
97  void PlayPreviousSong(PlayList<T>** currentSong){
98
99      if(*currentSong == nullptr){
100         std::cout<<"Playlist is empty."<<std::endl;
```

```
100             std::cout<<"Playlist is empty."<<std::endl;
101             return;
102         }
103         *currentSong = (*currentSong)->prev;
104         std::cout<<"Now Playing: "<< (*currentSong)->data<< std::endl;
105     }
106
107     //loop the PlayList
108     template <typename T>
109     void PlayinLoop(PlayList<T>* head){
110
111         if(head == nullptr){
112             std::cout<<"The Playlist is empty."<<std::endl;
113             return;
114         }
115         PlayList<T>* currentSong = head;
116         do{
117             std::cout<<"Playing: "<<currentSong->data<<std::endl;
118             currentSong = currentSong->next;
119         }while(currentSong != head);
120     }
121
122     //currently playing
123     template <typename T>
124     void PlayCurrentSong(PlayList<T>* currentSong){
125
126         if(currentSong == nullptr){
127             std::cout<<"Playlist is Empty"<<std::endl;
128             return;
129         }
130         std::cout << "Now Playing: "<<currentSong->data<< std::endl;
131     }
132
133     #endif
```

MyPlaylist.cpp

HOA 3.1 > MyPlaylist.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "playlist_ll.h"

int main(){
    PlayList<std::string>* head = nullptr;
    PlayList<std::string>* currentSong = nullptr;
    int choice;
    std::string Song;

    do{
        std::cout << "\n====== MY PLAYLIST ======\n";
        std::cout << "1. Add Song\n";
        std::cout << "2. Display Playlist\n";
        std::cout << "3. Play Current Song\n";
        std::cout << "4. Next Song\n";
        std::cout << "5. Previous Song\n";
        std::cout << "6. Delete Song\n";
        std::cout << "7. Play Playlist in Loop\n";
        std::cout << "8. Exit\n";
        std::cout << "Enter choice: ";
        std::cin >> choice;

        switch (choice){
        case 1:
            std::cout<<"Enter Song Title: ";
            std::getline(std::cin>>std::ws, Song);

            if(head == nullptr){
                head = CreatePlayList(Song);
                head->next = head;
                head->prev = head;
                currentSong = head;
            }else{
                InsertNewSong(Song, head);
            }

            std::cout<<"Song Added Successfully!\n";
            break;

        case 2:
            DisplayPlaylist(head);
            break;

        case 3:
            PlayCurrentSong(currentSong);
            break;

        case 4:
            PlayNextSong(&currentSong);
            break;
```

```
52
53          case 5:
54              PlayPreviousSong(&currentSong);
55              break;
56
57          case 6:
58              std::cout<<"Enter Song Title to Delete: ";
59              std::getline(std::cin>>std::ws, Song);
60
61              RemoveSong(Song, &head);
62
63              if(head != nullptr)
64                  currentSong = head;
65              else
66                  currentSong = nullptr;
67              std::cout<<"Song Deleted Successfully!\n";
68              break;
69
70          case 7:
71              PlayinLoop(head);
72              break;
73
74          case 8:
75              std::cout<<"Thank You for Listening!\n";
76              break;
77
78          default:
79              std::cout<<"Invalid choice!\n";
80          }
81      }while(choice != 8);
82      return 0;
83  }
```

Console

```
====== MY PLAYLIST ======
1. Add Song
2. Display Playlist
3. Play Current Song
4. Next Song
5. Previous Song
6. Delete Song
7. Play Playlist in Loop
8. Exit
Enter choice: 1
Enter Song Title: Sign of the Times
Song Added Successfully!
```

```
====== MY PLAYLIST ======
1. Add Song
2. Display Playlist
3. Play Current Song
4. Next Song
5. Previous Song
6. Delete Song
7. Play Playlist in Loop
8. Exit
Enter choice: 2
Sign of the Times
About You
Let it Happen
Opera House
Guilty as Sin?
```

```
====== MY PLAYLIST ======
1. Add Song
2. Display Playlist
3. Play Current Song
4. Next Song
5. Previous Song
6. Delete Song
7. Play Playlist in Loop
8. Exit
Enter choice: 3
Now Playing: Sign of the Times
```

```
====== MY PLAYLIST ======
1. Add Song
2. Display Playlist
3. Play Current Song
4. Next Song
5. Previous Song
6. Delete Song
7. Play Playlist in Loop
8. Exit
Enter choice: 4
Now Playing: About You
```

```
====== MY PLAYLIST ======
1. Add Song
2. Display Playlist
3. Play Current Song
4. Next Song
5. Previous Song
6. Delete Song
7. Play Playlist in Loop
8. Exit
Enter choice: 5
Now Playing: Sign of the Times
```

```
====== MY PLAYLIST ======
1. Add Song
2. Display Playlist
3. Play Current Song
4. Next Song
5. Previous Song
6. Delete Song
7. Play Playlist in Loop
8. Exit
Enter choice: 6
Enter Song Title to Delete: About You
Song Deleted Successfully!
```

```
====== MY PLAYLIST ======
1. Add Song
2. Display Playlist
3. Play Current Song
4. Next Song
5. Previous Song
6. Delete Song
7. Play Playlist in Loop
8. Exit
Enter choice: 7
Playing: Sign of the Times
Playing: Let it Happen
Playing: Opera House
Playing: Guilty as Sin?
```

```
====== MY PLAYLIST ======
1. Add Song
2. Display Playlist
3. Play Current Song
4. Next Song
5. Previous Song
6. Delete Song
7. Play Playlist in Loop
8. Exit
Enter choice: 8
Thank You for Listening!
```

Observations:

This code demonstrates that a circular doubly linked list is an effective data structure for managing a music playlist. It allows users to navigate seamlessly between songs, add and remove songs, and continuously loop through the playlist. The modular design also improves code organization by separating playlist operations from the user interface, making the program easier to understand and maintain.

Conclusion:

In this activity, I gained a better understanding of how circular doubly linked lists work and how they can be applied to real-world programs such as a music playlist. I learned how to dynamically add, delete, and traverse nodes while maintaining the connections between them. I also improved my understanding of pointer manipulation, memory management, and modular programming by separating the playlist operations from the main program. Overall, this project strengthened my knowledge of linked lists and demonstrated their advantages in implementing applications that require efficient navigation.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** _____

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** _____

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** _____

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.